

# Inside DataLens.

## Architecture, methods, and a live component audit.

A walkthrough of every analytical module in the project, the algorithms it relies on, and what each one produced when exercised against 3 representative datasets.

| DATASETS | COMPONENTS | PASS | PASS RATE |
|----------|------------|------|-----------|
| 3        | 72         | 72   | 100.0%    |

## Abstract

DataLens AI is a Python-first analytical pipeline that turns an arbitrary tabular dataset into a complete dashboard, a typeset PDF report, and a chat-style question-answering interface, all without the user having to write code. This whitepaper documents how it works, end to end, with two goals in mind. First, to give a self-contained explanation of every analytical module — what it computes, why that computation matters, and which classical algorithm it sits on top of. Second, to back those explanations with a live audit: every module described here was exercised against three representative datasets just before this document was generated, and the empirical evidence (timings, summary metrics, and any failures) is included verbatim.

The audience for this document is anyone who wants to understand DataLens deeply enough to trust it: a stakeholder evaluating its outputs, an engineer debugging or extending it, or a reviewer assessing whether its methodology is sound. We have therefore traded brevity for clarity: each chapter starts with a primer on the underlying concepts (e.g. what SHAP values are and what they are useful for) before showing the live results.

**Reproducibility note.** Re-running `python tools/component_test.py` regenerates the manifest; re-running `python tools/whitepaper_builder.py` regenerates this PDF.

# Contents

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| <b>Abstract</b>                                                   | <b>2</b>  |
| <b>1 Introduction</b>                                             | <b>4</b>  |
| <b>2 System Architecture</b>                                      | <b>4</b>  |
| 2.1 Top-down view . . . . .                                       | 4         |
| 2.2 Phase orchestration . . . . .                                 | 5         |
| <b>3 Ingest — understanding the dataset</b>                       | <b>5</b>  |
| 3.1 Live evidence . . . . .                                       | 5         |
| <b>4 Quality — can we trust the data?</b>                         | <b>7</b>  |
| 4.1 Live evidence . . . . .                                       | 8         |
| 4.2 Visual gallery . . . . .                                      | 9         |
| <b>5 Statistics — pairwise structure and outliers</b>             | <b>9</b>  |
| 5.1 Live evidence . . . . .                                       | 10        |
| 5.2 Visual gallery . . . . .                                      | 10        |
| <b>6 Modeling — baselines, leaderboard, and explainability</b>    | <b>10</b> |
| 6.1 Live evidence . . . . .                                       | 12        |
| 6.2 Visual gallery . . . . .                                      | 12        |
| <b>7 Time-series &amp; text — temporal structure and language</b> | <b>13</b> |
| 7.1 Live evidence . . . . .                                       | 14        |
| 7.2 Visual gallery . . . . .                                      | 14        |
| <b>8 Drift — has the data changed?</b>                            | <b>15</b> |
| 8.1 Live evidence . . . . .                                       | 15        |
| <b>9 Reporting — charts and the PDF whitepaper</b>                | <b>16</b> |
| 9.1 Live evidence . . . . .                                       | 16        |

|                                                                  |           |
|------------------------------------------------------------------|-----------|
| 9.2 Visual gallery . . . . .                                     | 17        |
| <b>10 Agentic — payload, brief, retrieval, and the LLM agent</b> | <b>18</b> |
| 10.1 Live evidence . . . . .                                     | 19        |
| <b>11 Timing budget</b>                                          | <b>19</b> |
| <b>12 Limitations and future work</b>                            | <b>20</b> |
| <b>References</b>                                                | <b>21</b> |
| <b>A Per-component test table</b>                                | <b>21</b> |
| <b>B Failure traces</b>                                          | <b>25</b> |

# 1 Introduction

DataLens has a deliberately uncluttered shape. A dataset arrives at `POST /api/analyze`; in  $\sim 5\text{--}15$  seconds a fully-analysed JSON payload comes back, populated with everything the dashboard needs and a `downloadLinks.report` URL pointing at a generated PDF. Every step in between is a small, individually-testable Python module.

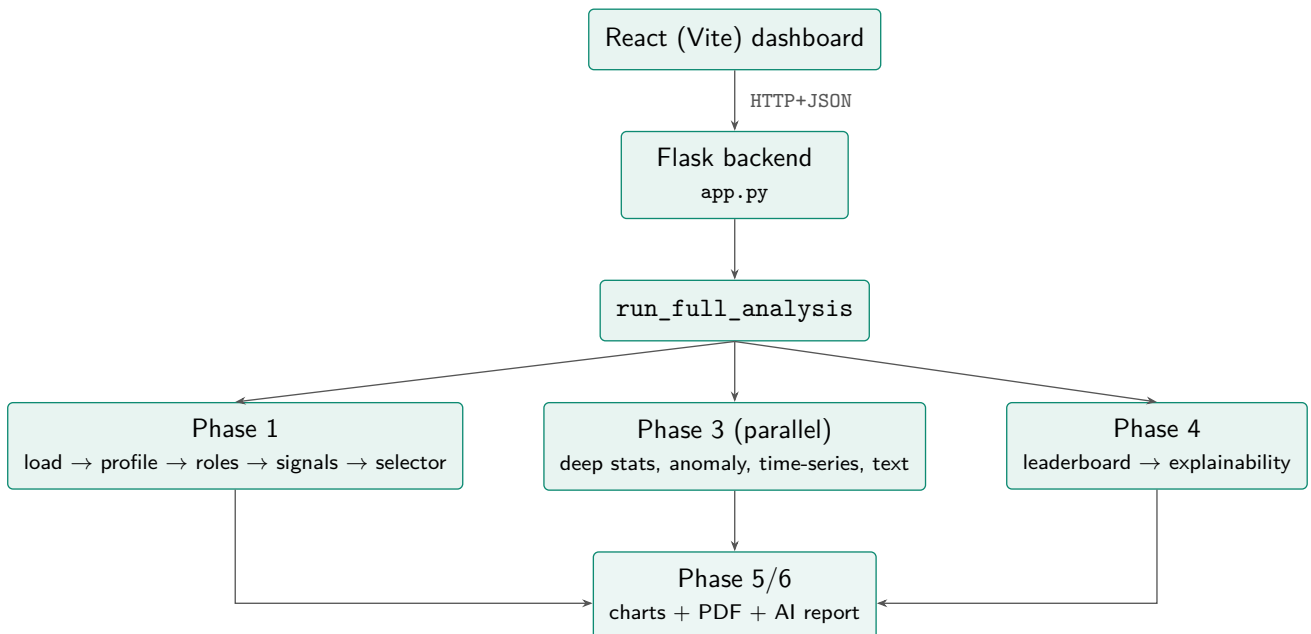
This whitepaper is structured as follows. Section 2 covers the overall architecture — the orchestrator, parallelism, and how phases share state. Sections 3–10 each pair a methods primer with the live evidence emitted by the corresponding module(s) when they were last exercised. Section 11 aggregates timing budgets across the test datasets. Section 12 is honest about what the project does *not* do well today.

The live evidence in this document came from 3 datasets: classification, regression, `no_target`.

## 2 System Architecture

### 2.1 Top-down view

DataLens is a single-process Flask backend that runs an analytical pipeline, plus a Vite-served React dashboard. The two communicate over a stable JSON contract. There is no database — every analysis lives in an in-memory cache keyed by a per-run `dataset_id` and is regenerated on demand.



**Figure 1:** High-level pipeline. A request travels from the React dashboard through `app.py` into `run_full_analysis`, which fans out across the six phases. Phase 3 runs in parallel via a `ThreadPoolExecutor`; the other phases are sequential because they either share state or rely on `matplotlib` (which is not thread-safe).

## 2.2 Phase orchestration

The orchestrator is `modules/pipeline.py:run_full_analysis()`. It runs in six phases, with two important properties. First, every phase is *wrapped* in `_safe_call()` which converts any exception into a JSON-safe error dict — a single failing module cannot sink the run. Second, phases that are pure-compute and independent are run in parallel using a `ThreadPoolExecutor` (default 4 workers); phases that are sequential or matplotlib-bound are not. The full timing breakdown is returned in `result["timings_ms"]` and visualised in the dashboard.

## 3 Ingest — understanding the dataset

The first job of a data-analysis pipeline is to figure out what kind of data it is looking at. DataLens does this in five short, sequential phases: load, profile, role inference, signal detection, and analysis selection.

**Loading** normalises raw bytes into a `pandas.DataFrame`. The loader handles CSV, TSV, Excel and JSON formats, sniffs separators, and tolerates BOMs. It is intentionally tolerant — bad rows are dropped with a warning rather than crashing the run.

**Profiling** captures the structural footprint of the dataset: shape, dtypes, per-column missing counts, per-column dtype histograms, sample of the head/tail, duplicate-row count, and cardinality. This is the cheapest and most general view, and almost every downstream module reads from it.

**Column role inference** assigns multi-label tags to each column (`numeric`, `categorical_low`, `id_like`, `price_like`, `volume_like`, `boolean`, `datetime_like`, `long_text`, etc.) using a mix of dtype inspection, name regexes (`price`, `revenue`, `id`, `date`), cardinality ratios and content sniffing. Roles are what allow the chart planner and the auto-cleaner to behave intelligently without being told column-by-column what to do.

**Signal detection** surfaces a dictionary of high-level dataset traits (`has_text_columns`, `has_long_text_columns`, `is_wide_dataset`, `is_imbalanced`, `has_potential_leakage`, etc.) which the analysis selector uses to decide which expensive analyses are worth running.

**Analysis selection** maps the (signals, mode, target column) triple onto a curated catalogue of analyses with declared requirements. The selector accepts, recommends, or skips each analysis; this is what differentiates the four QUICK/STANDARD/DEEP/RESEARCH modes.

### 3.1 Live evidence

We probed every module in the Ingest category against three demo datasets. The table below shows each component's availability flag, runtime, and one-line summary. Failures (if any) are listed by name in the appendix.

**Dataset: classification**

5,060 rows · 21 cols · target = churned

| MODULE            | STATUS | AVAIL. | TIME  | SUMMARY                                                                                                                                                                  |
|-------------------|--------|--------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| loader            | ok     | –      | 0 ms  | loaded 5,060 rows × 21 cols from customer_churn.csv                                                                                                                      |
| profiler          | ok     | –      | 21 ms | profile: 9 numeric, 12 categorical, 2 date · 4,763 missing cells                                                                                                         |
| column_roles      | ok     | –      | 8 ms  | roles: id=0, price=['monthly_charges', 'total_charges', 'feedback_text'], volume=[], target_candidates=['gender', 'region', 'plan', 'contract_type', 'payment_method'... |
| dataset_signals   | ok     | –      | 34 ms | signals on: has_numeric_columns, has_multiple_numeric_columns, has_categorical_columns, has_datetime_columns, has_text_columns, has_long_text_columns                    |
| analysis_selector | ok     | –      | 34 ms | selected=27 · recommended=0 · skipped=1                                                                                                                                  |

**Dataset: regression**

6,035 rows · 18 cols · target = price

| MODULE            | STATUS | AVAIL. | TIME  | SUMMARY                                                                                                                                                                              |
|-------------------|--------|--------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| loader            | ok     | –      | 0 ms  | loaded 6,035 rows × 18 cols from real_estate.csv                                                                                                                                     |
| profiler          | ok     | –      | 18 ms | profile: 9 numeric, 9 categorical, 1 date · 650 missing cells                                                                                                                        |
| column_roles      | ok     | –      | 6 ms  | roles: id=0, price=['days_on_market', 'price'], volume=['lot_size'], target_candidates=['neighborhood', 'property_type', 'zoning', 'bedrooms', 'floors', 'has_garage', 'has_pool'... |
| dataset_signals   | ok     | –      | 28 ms | signals on: has_numeric_columns, has_multiple_numeric_columns, has_categorical_columns, has_datetime_columns, has_text_columns, has_long_text_columns                                |
| analysis_selector | ok     | –      | 28 ms | selected=20 · recommended=0 · skipped=8                                                                                                                                              |

**Dataset: no\_target**

4,035 rows · 22 cols · target = none

| MODULE            | STATUS | AVAIL. | TIME  | SUMMARY                                                                                                                                                                         |
|-------------------|--------|--------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| loader            | ok     | –      | 0 ms  | loaded 4,035 rows × 22 cols from medical_records.csv                                                                                                                            |
| profiler          | ok     | –      | 18 ms | profile: 12 numeric, 10 categorical, 1 date · 905 missing cells                                                                                                                 |
| column_roles      | ok     | –      | 7 ms  | roles: id=0, price=['heart_rate'], volume=['medication_count'], target_candidates=['gender', 'ethnicity', 'diagnosis_code', 'diagnosis_label', 'smoker', 'medication_count']... |
| dataset_signals   | ok     | –      | 29 ms | signals on: has_numeric_columns, has_multiple_numeric_columns, has_categorical_columns, has_datetime_columns, has_text_columns, has_missing_values                              |
| analysis_selector | ok     | –      | 28 ms | selected=20 · recommended=0 · skipped=8                                                                                                                                         |

## 4 Quality — can we trust the data?

DataLens computes two complementary scores out of 100: a *health score* that reflects how clean the data is on its own terms, and an *ML readiness score* that reflects how easily the data could be modeled.

**Health score** blends four sub-scores: *completeness* (1 minus mean missing rate), *consistency* (penalty for mixed types and parse failures), *uniqueness* (penalty for duplicate rows), and *outlier safety* (share of numeric values inside the  $1.5 \times \text{IQR}$  fences). The four are then weighted and combined; thresholds at 60 / 80 / 95 give POOR / GOOD / EXCELLENT.

**ML readiness** reflects something different: even a clean dataset can be hard to model (too few rows, too many constant columns, severe class imbalance). The ML readiness score weighs the row/column ratio, fraction of usable predictive columns, class balance for any candidate target, and target stability.

**Auto-cleaner** runs a sequence of conservative repair actions: trim whitespace, normalise booleans, drop fully-empty columns, drop fully-duplicated rows, fill numeric columns with the median, fill categoricals with a sentinel, and parse date-like strings. Every action is recorded with a risk tag (*low/medium/high*) so the cleaning summary can always be audited.

**Advanced indicators** add three secondary signals on top: *column utility* (a 0–100 score per column that combines variance, missingness, and predictive shape), a quick *IQR-based anomaly score* per row (used as a coarse indicator pre-ensemble), and a *freshness* estimate (how recent the latest dates are).



**Signal engine** is a rules layer that converts those indicators into human-readable signals with severity (**low/medium/high/critical**), evidence and a recommendation, e.g. “*column revenue is 23% missing — review imputation strategy or drop rows for revenue-anchored analyses.*”

## 4.1 Live evidence

We probed every module in the Quality category against three demo datasets. The table below shows each component’s availability flag, runtime, and one-line summary. Failures (if any) are listed by name in the appendix.

### Dataset: classification

5,060 rows · 21 cols · target = churned

| MODULE              | STATUS | AVAIL. | TIME  | SUMMARY                                              |
|---------------------|--------|--------|-------|------------------------------------------------------|
| health_score        | ok     | –      | 24 ms | health = 98.21/100 (Excellent)                       |
| ml_readiness        | ok     | –      | 2 ms  | ML readiness = 74.33/100 (Mostly Ready)              |
| advanced_indicators | ok     | –      | 58 ms | utility ranked 21 cols · top anomaly score 0.00      |
| signal_engine       | ok     | –      | 83 ms | 8 signals · Low:3, None:1, Medium:2, Informational:2 |
| cleaner             | ok     | –      | 86 ms | cleaning 98.21 → 99.78 · actions=7                   |

### Dataset: regression

6,035 rows · 18 cols · target = price

| MODULE              | STATUS | AVAIL. | TIME   | SUMMARY                                              |
|---------------------|--------|--------|--------|------------------------------------------------------|
| health_score        | ok     | –      | 21 ms  | health = 99.52/100 (Excellent)                       |
| ml_readiness        | ok     | –      | 2 ms   | ML readiness = 80.82/100 (Mostly Ready)              |
| advanced_indicators | ok     | –      | 161 ms | utility ranked 18 cols · top anomaly score 0.00      |
| signal_engine       | ok     | –      | 184 ms | 8 signals · Low:4, None:1, Medium:1, Informational:2 |
| cleaner             | ok     | –      | 79 ms  | cleaning 99.52 → 99.79 · actions=4                   |

### Dataset: no\_target

4,035 rows · 22 cols · target = none

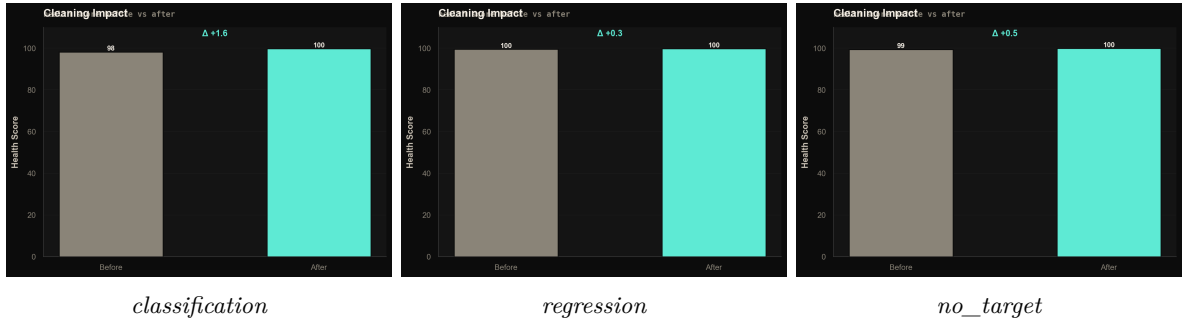
| MODULE       | STATUS | AVAIL. | TIME  | SUMMARY                                 |
|--------------|--------|--------|-------|-----------------------------------------|
| health_score | ok     | –      | 25 ms | health = 99.42/100 (Excellent)          |
| ml_readiness | ok     | –      | 2 ms  | ML readiness = 78.11/100 (Mostly Ready) |

|                     |    |   |       |                                                      |
|---------------------|----|---|-------|------------------------------------------------------|
| advanced_indicators | ok | — | 43 ms | utility ranked 22 cols · top anomaly score 0.00      |
| signal_engine       | ok | — | 66 ms | 8 signals · Low:4, None:1, Medium:1, Informational:2 |
| cleaner             | ok | — | 91 ms | cleaning 99.42 → 99.89 · actions=5                   |

## 4.2 Visual gallery



**Figure 2: Dataset Health Components** — Completeness, consistency, uniqueness, and outlier safety.



**Figure 3: Cleaning Impact** — Quality score before and after cleaning.

## 5 Statistics — pairwise structure and outliers

Once the dataset is understood and scored, DataLens runs two heavier modules: *deep statistics* and an *anomaly ensemble*.

**Deep statistics v2** computes per-column descriptive moments (mean, median, std, skew, kurtosis), classical pairwise tests, and bivariate relationship strength. For *numeric-numeric* pairs we compute Pearson’s  $r$  ( $\rho_{XY} = \text{Cov}(X, Y) / (\sigma_X \sigma_Y)$ ) and report it whenever both sides are continuous. For *categorical-categorical* pairs we compute Cramér’s  $V$  [11], a normalisation of the chi-squared statistic that is bounded in  $[0, 1]$ . For *categorical-numeric* pairs with a binary categorical we compute the *point-biserial correlation*; for higher-cardinality categoricals we run a non-parametric group-difference test (Kruskal-Wallis) and report the effect size  $\eta^2$ .

All bivariate tests share a small budget (**max\_pairs**) that scales with mode, so the runtime stays bounded even on wide datasets. The output is sorted by absolute effect size — the biggest relationships float to the top of the bivariate table.

**Anomaly ensemble** blends three independent detectors: *Isolation Forest* [3], *Local Outlier*

*Factor* [4], and a *robust z-score* based on the median absolute deviation. Each detector returns a per-row score in  $[0, 1]$ ; the ensemble averages them after rank-normalisation so the final score is robust to any one detector misbehaving on the dataset’s geometry. Rows with the highest combined scores are surfaced — both for inspection and to feed the anomaly score chart in the dashboard.

## 5.1 Live evidence

We probed every module in the Statistics category against three demo datasets. The table below shows each component’s availability flag, runtime, and one-line summary. Failures (if any) are listed by name in the appendix.

### Dataset: classification

5,060 rows · 21 cols · target = churned

| MODULE             | STATUS | AVAIL. | TIME   | SUMMARY                                    |
|--------------------|--------|--------|--------|--------------------------------------------|
| deep_statistics_v2 | ok     | yes    | 2.78 s | pairs: num=20, cat=20, groupdiff=20        |
| anomaly_ensemble   | ok     | yes    | 1.31 s | available=True · top rows=25 · detectors=0 |

### Dataset: regression

6,035 rows · 18 cols · target = price

| MODULE             | STATUS | AVAIL. | TIME   | SUMMARY                                    |
|--------------------|--------|--------|--------|--------------------------------------------|
| deep_statistics_v2 | ok     | yes    | 3.35 s | pairs: num=20, cat=15, groupdiff=20        |
| anomaly_ensemble   | ok     | yes    | 428 ms | available=True · top rows=25 · detectors=0 |

### Dataset: no\_target

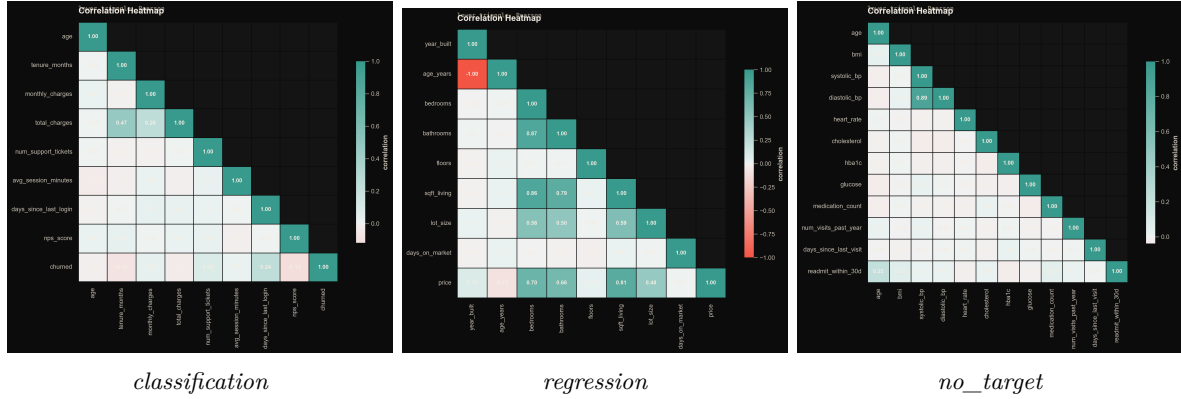
4,035 rows · 22 cols · target = none

| MODULE             | STATUS | AVAIL. | TIME   | SUMMARY                                    |
|--------------------|--------|--------|--------|--------------------------------------------|
| deep_statistics_v2 | ok     | yes    | 2.02 s | pairs: num=20, cat=20, groupdiff=20        |
| anomaly_ensemble   | ok     | yes    | 522 ms | available=True · top rows=25 · detectors=0 |

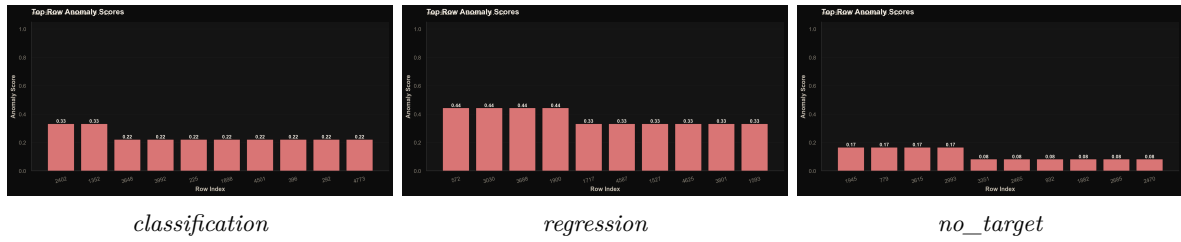
## 5.2 Visual gallery

## 6 Modeling — baselines, leaderboard, and explainability

When a target column is supplied, DataLens runs a small but principled modeling pipeline. The goal is not to crown a state-of-the-art model: it is to give the analyst an honest baseline plus an explanation of what is driving it.



**Figure 4: Correlation Heatmap** — Pairwise relationships between numeric columns.



**Figure 5: Top Anomaly Scores** — Rows with highest anomaly scores.

**Preprocessing** happens once, in a `ColumnTransformer`. Numeric columns are imputed with the median and standardised. Categorical columns are imputed with a constant sentinel ('`__missing__`') and one-hot encoded; before encoding they are stringified, which avoids the classic crash where mixed bool/string categoricals trip up scikit-learn.

**Leaderboard** fits a curated list of estimators (Logistic Regression, Random Forest, Gradient Boosting, Ridge / Lasso for regression, plus a `DummyClassifier` or `DummyRegressor` as a sanity floor). Each estimator is evaluated with stratified  $k$ -fold cross-validation (or plain  $k$ -fold for regression). The primary metric is  $F_1$ -weighted for classification and  $R^2$  for regression. The winner is then refit on the full training set and tested on a held-out slice.

**SHAP explainability.** For the leaderboard winner, DataLens produces a global feature importance ranking using SHAP values. SHAP (Shapley Additive exPlanations) [1] is a unified framework borrowed from cooperative game theory [12]. For a model  $f$  and an input  $x = (x_1, \dots, x_p)$ , the SHAP value for feature  $i$  is the average marginal contribution that feature makes to the prediction, across every possible ordering of the other features:

$$\phi_i(f, x) = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|! (|F| - |S| - 1)!}{|F|!} [f(S \cup \{i\}) - f(S)].$$

Three properties make SHAP appealing in production: *local accuracy* (the SHAP values of the features sum to the model's prediction), *missingness* (a feature absent from the input contributes zero), and *consistency* (if a model is changed so a feature contributes more, that feature's SHAP value cannot decrease). DataLens uses `shap.LinearExplainer` for linear leaders and `shap.TreeExplainer` for tree leaders, and falls back to *permutation importance* [2] when SHAP

cannot run. The mean of  $|\phi_i|$  across the validation set is what gets plotted as global feature importance.

**What it is useful for.** Global SHAP rankings answer “*which features drive the model’s decisions on average?*”, which is the question a non-ML stakeholder always asks first. Per-row SHAP values (also produced by DataLens) answer the harder version: “*why did this specific customer get this specific score?*” — the answer that auditing, fairness review, and customer support all need.

## 6.1 Live evidence

We probed every module in the Modeling category against three demo datasets. The table below shows each component’s availability flag, runtime, and one-line summary. Failures (if any) are listed by name in the appendix.

### Dataset: classification

5,060 rows · 21 cols · target = churned

| MODULE            | STATUS | AVAIL. | TIME   | SUMMARY                                                                                         |
|-------------------|--------|--------|--------|-------------------------------------------------------------------------------------------------|
| model_leaderboard | ok     | yes    | 7.44 s | task=classification · winner=LogisticRegression · score=0.8745                                  |
| explainability    | ok     | yes    | 7.84 s | method=shap.LinearExplainer · top features: feedback_text, days_since_last_login, contract_type |

### Dataset: regression

6,035 rows · 18 cols · target = price

| MODULE            | STATUS | AVAIL. | TIME   | SUMMARY                                    |
|-------------------|--------|--------|--------|--------------------------------------------|
| model_leaderboard | ok     | yes    | 385 ms | task=regression · winner=None · score=None |
| explainability    | ok     | no     | 239 ms | skipped — no leaderboard winner            |

### Dataset: no\_target

4,035 rows · 22 cols · target = none

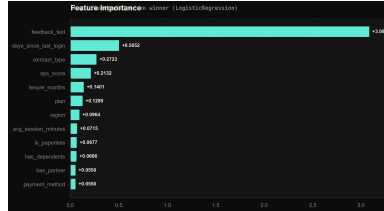
| MODULE            | STATUS | AVAIL. | TIME | SUMMARY                             |
|-------------------|--------|--------|------|-------------------------------------|
| model_leaderboard | ok     | no     | 0 ms | skipped — no target column provided |
| explainability    | ok     | no     | 0 ms | skipped — no target column provided |

## 6.2 Visual gallery



classification

**Figure 6: Model Leaderboard** — Cross-validated model scores. Winner highlighted in teal.



classification

**Figure 7: Feature Importance** — Mean |SHAP| or permutation importance from the leaderboard winner.

## 7 Time-series & text — temporal structure and language

Two analytical lenses come into play when a dataset has structured time or free text.

**Time-series detection.** DataLens scores each column for date-likeness (name regex, parse rate, monotonicity, cardinality) and picks the highest-scoring column. It then estimates the dominant frequency from the median spacing between observations (seconds → minutes → hours → days → weeks → months → years).

**Stationarity.** Two complementary tests are run on the paired numeric series: the augmented Dickey–Fuller (ADF) test [9], where rejecting the null means the series *is* stationary, and the KPSS test [10], where rejecting the null means it is *not* stationary. Cross-checking ADF and KPSS gives a four-way verdict (*stationary*, *trend-stationary*, *difference-stationary*, *inconclusive*) that is more honest than either test on its own.

**Decomposition.** If the inferred period is at least 2 and the series has  $\geq 2 \times \text{period} + 4$  observations, DataLens runs an STL decomposition [5]. The strength of trend  $T$  and seasonality  $S$  are reported using the standard formulation [6]:  $T = \max(0, 1 - \text{Var}(R)/\text{Var}(T + R))$ , where  $R$  is the residual.

**Forecast.** A Holt–Winters exponential-smoothing model is fit on the first 80% of the series and validated on the held-out tail. A naive last-value baseline is reported alongside so the analyst can see how much lift the seasonal model adds.

**Text profiling.** Columns whose dtype is object/string are screened with three light heuristics: at least 30% of non-empty values must contain  $\geq 2$  tokens, the mean character length must be  $\geq 8$ , and the column must not look ID-like. For each surviving column, DataLens computes basic length statistics, total tokens, vocabulary size, type–token ratio, mean sentence length, top

unigrams and bigrams (stop-words filtered when NLTK is available), and regex pattern hits for emails, URLs, phone numbers, mentions and hashtags.

## 7.1 Live evidence

We probed every module in the Time & Text category against three demo datasets. The table below shows each component's availability flag, runtime, and one-line summary. Failures (if any) are listed by name in the appendix.

### Dataset: classification

5,060 rows · 21 cols · target = churned

| MODULE       | STATUS | AVAIL. | TIME   | SUMMARY                                                     |
|--------------|--------|--------|--------|-------------------------------------------------------------|
| time_series  | ok     | yes    | 177 ms | date=num_support_tickets · numeric=churned · freq=irregular |
| text_profile | ok     | yes    | 75 ms  | profiled 1 text col(s): feedback_text                       |

### Dataset: regression

6,035 rows · 18 cols · target = price

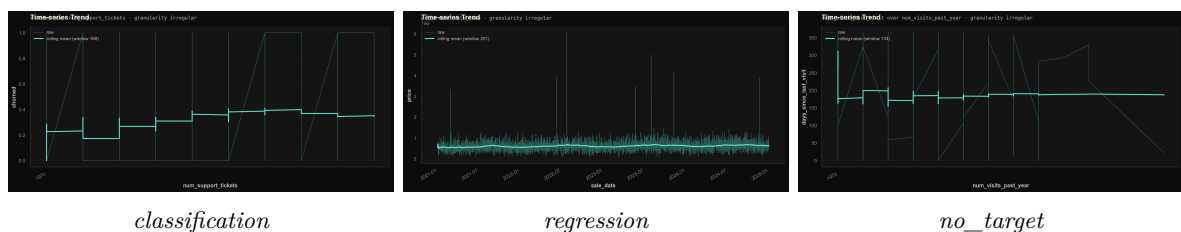
| MODULE       | STATUS | AVAIL. | TIME   | SUMMARY                                         |
|--------------|--------|--------|--------|-------------------------------------------------|
| time_series  | ok     | yes    | 337 ms | date=sale_date · numeric=price · freq=irregular |
| text_profile | ok     | yes    | 81 ms  | profiled 1 text col(s): description             |

### Dataset: no\_target

4,035 rows · 22 cols · target = none

| MODULE       | STATUS | AVAIL. | TIME   | SUMMARY                                                                    |
|--------------|--------|--------|--------|----------------------------------------------------------------------------|
| time_series  | ok     | yes    | 562 ms | date=num_visits_past_year · numeric=days_since_last_visit · freq=irregular |
| text_profile | ok     | yes    | 38 ms  | profiled 1 text col(s): clinical_notes                                     |

## 7.2 Visual gallery



**Figure 8: Time-series Trend** — Trend of 'days\_since\_last\_visit' over 'num\_visits\_past\_year'.

## 8 Drift — has the data changed?

Drift answers a deceptively simple question: *is the new data behaving like the old data?* Production ML systems quietly degrade because the world the model was trained on no longer matches the world it is now scoring against. Detecting this early is what turns model monitoring into model maintenance.

**PSI for numeric columns.** The Population Stability Index [7] bins the reference and current arrays using the reference’s quantiles, then sums  $\sum_b (p_{\text{cur},b} - p_{\text{ref},b}) \log(p_{\text{cur},b}/p_{\text{ref},b})$  across bins. Conventional cut-offs are 0.10 for *minor*, 0.25 for *moderate*, and anything above for *severe* drift.

**KS test.** The two-sample Kolmogorov–Smirnov statistic [8] complements PSI. PSI is decomposable per-bin and easy to communicate to stakeholders; KS is more sensitive to subtle distribution shifts.

**PSI for categoricals.** For categorical columns we compute the same divergence using observed frequencies, which makes PSI a natural drop-in for both numeric and categorical variables.

**Chi-squared test.** For categoricals we additionally run a contingency chi-squared test, which is the natural pair to PSI when one wants a *p*-value rather than a magnitude.

**Verdict.** The per-column severities (*stable*, *minor*, *moderate*, *severe*) are aggregated into an overall verdict, sorted, and surfaced in the drift tab and PDF.

### 8.1 Live evidence

We probed every module in the Drift category against three demo datasets. The table below shows each component’s availability flag, runtime, and one-line summary. Failures (if any) are listed by name in the appendix.

**Dataset: classification**

5,060 rows · 21 cols · target = churned

| MODULE         | STATUS | AVAIL. | TIME   | SUMMARY                                                |
|----------------|--------|--------|--------|--------------------------------------------------------|
| drift_analysis | ok     | yes    | 4.46 s | shared cols=21 · drifted=4 · severe=3 · verdict=severe |

**Dataset: regression**

6,035 rows · 18 cols · target = price

| MODULE         | STATUS | AVAIL. | TIME   | SUMMARY                                                |
|----------------|--------|--------|--------|--------------------------------------------------------|
| drift_analysis | ok     | yes    | 7.16 s | shared cols=18 · drifted=3 · severe=3 · verdict=severe |



**Dataset:** no\_target

4,035 rows · 22 cols · target = none

| MODULE         | STATUS | AVAIL. | TIME   | SUMMARY                                                |
|----------------|--------|--------|--------|--------------------------------------------------------|
| drift_analysis | ok     | yes    | 2.66 s | shared cols=22 · drifted=2 · severe=2 · verdict=severe |

## 9 Reporting — charts and the PDF whitepaper

DataLens produces two visual deliverables: a static chart gallery rendered into PNGs that the dashboard mounts via `/static/charts/`, and a fully-typeset PDF report that mirrors the editorial dark theme of the dashboard.

**Chart planner.** The planner is a pure function. It receives the profile, health, advanced, cleaning, column roles, and an optional `context` carrying upstream phases (target column, leaderboard, explainability, time-series result, deep statistics). It returns a list of  $\sim 20+$  chart specs, each a small dict with `type`, `title`, `column(s)` and a `reason` string. Charts are only proposed when their underlying data exists, so a dataset without a target column does not produce target-aware charts; a dataset without a date column does not produce a time-series chart, and so on.

**Chart renderer.** The visualizer walks the plan and dispatches each spec to a dedicated matplotlib + seaborn renderer. Every chart shares the editorial palette (cream + teal on near-black) so the gallery and the PDF feel like one product. New chart types include *schema composition* (donut), *cardinality strip*, *violin*, *numeric overview* (top-6 histograms in one figure), *Pareto categorical*, *top correlations bar*, *target distribution*, *target vs feature*, *leaderboard bars*, *feature importance bars*, *time-series trend with rolling and STL overlay*, and *bivariate highlights*.

**PDF builder.** The PDF report is rendered with `matplotlib.backends.backend_pdf.PdfPages` so the project never needs a heavy LaTeX or HTML-to-PDF dependency. Pages flow as: cover (gradient hero band, KPI tiles), contents, executive summary, quality, signals, cleaning, deep statistics, modeling, time-series and text, AI narrative, and finally a *visual evidence* section with two charts per page. Section dividers are colour-coded by category, and every body page carries a branded header and a paginated footer.

### 9.1 Live evidence

We probed every module in the Reporting category against three demo datasets. The table below shows each component's availability flag, runtime, and one-line summary. Failures (if any) are listed by name in the appendix.

**Dataset:** classification

5,060 rows · 21 cols · target = churned

| MODULE             | STATUS | AVAIL. | TIME   | SUMMARY                                                          |
|--------------------|--------|--------|--------|------------------------------------------------------------------|
| chart_planner      | ok     | —      | 31 ms  | planned 25 charts (22 distinct types)                            |
| visualizer         | ok     | —      | 0 ms   | rendered 23 charts to /static/charts/                            |
| pdf_report_builder | ok     | —      | 1.80 s | PDF written:<br>comp_test_classification_report.pdf<br>(1366 KB) |

**Dataset: regression**

6,035 rows · 18 cols · target = price

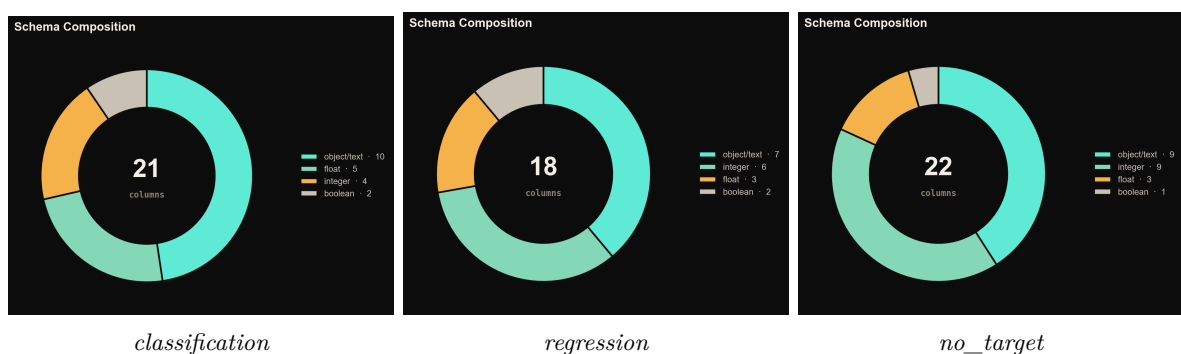
| MODULE             | STATUS | AVAIL. | TIME   | SUMMARY                                                   |
|--------------------|--------|--------|--------|-----------------------------------------------------------|
| chart_planner      | ok     | —      | 23 ms  | planned 23 charts (19 distinct types)                     |
| visualizer         | ok     | —      | 0 ms   | rendered 21 charts to /static/charts/                     |
| pdf_report_builder | ok     | —      | 1.58 s | PDF written:<br>comp_test_regression_report.pdf (1237 KB) |

**Dataset: no\_target**

4,035 rows · 22 cols · target = none

| MODULE             | STATUS | AVAIL. | TIME   | SUMMARY                                                  |
|--------------------|--------|--------|--------|----------------------------------------------------------|
| chart_planner      | ok     | —      | 23 ms  | planned 20 charts (17 distinct types)                    |
| visualizer         | ok     | —      | 0 ms   | rendered 19 charts to /static/charts/                    |
| pdf_report_builder | ok     | —      | 1.51 s | PDF written:<br>comp_test_no_target_report.pdf (1223 KB) |

## 9.2 Visual gallery

**Figure 9: Schema Composition** — Distribution of column dtypes in the dataset.



no\_target

**Figure 10:** *Numeric Columns at a Glance* — Compact 6-up overview of the most analytically useful numerics.

## 10 Agentic — payload, brief, retrieval, and the LLM agent

The final layer of DataLens turns analytical artefacts into a conversational interface that an end user can actually talk to.

**Dashboard payload.** The frontend API takes the heavy pipeline result and projects it into a flat, JSON-safe payload that the React app consumes. The payload is camelCased and includes a small set of frontend-only conveniences (`downloadLinks`, `distribution`, `metrics`) that the dashboard cards depend on.

**Dataset brief.** The agent brief is a ~9 KB JSON dictionary carefully packed to fit inside an LLM context window. It contains: identity, schema (with cardinalities and missing-rate ranking), quality, signals, ML headlines (leaderboard winner, top SHAP features, VIF, leakage), time/text/drift, top correlations, sample rows, and the AI narrative. The brief is what gives the model accurate, *numeric* grounding when it answers questions like “*what is the correlation between tenure and total charges?*”

**RAG fact index.** On top of the brief, DataLens flattens the analysis result into a list of small **Fact** objects (`(path, value)`). A retrieval module embeds them with Ollama’s `nomic-embed-text` when available, or falls back to TF-IDF if not. Cosine similarity selects the top-*k* facts at query time and the writer prompt ingests them as the canonical evidence block.

**Agent loop.** The agent runs a planner → executor → critic → writer cycle. The planner proposes which tools to call (`get_dataset_brief`, `rank_columns_by_missing`, `get_anomaly_top_rows`, `get_leaderboard`, etc.); the executor calls them; the critic decides if the evidence is sufficient; the writer composes the markdown answer. A *fast-path* variant skips the planner and critic and runs a single writer call — this is what the Ask Anything panel uses by default to keep the conversation snappy.

**Backends.** The model router prefers a local Ollama daemon. `gpt-oss:120b-cloud` on Ollama Cloud’s free tier is a strong fast default, `qwen3:4b` is the small local-first model, and OpenRouter is the third fallback when Ollama is unreachable. If all three fail, DataLens uses a deterministic heuristic writer that surfaces the retrieved facts directly — the answer is honestly labelled “*no LLM endpoint reachable.*”

## 10.1 Live evidence

We probed every module in the Agentic category against three demo datasets. The table below shows each component's availability flag, runtime, and one-line summary. Failures (if any) are listed by name in the appendix.

### Dataset: classification

5,060 rows · 21 cols · target = churned

| MODULE       | STATUS | AVAIL. | TIME   | SUMMARY                                         |
|--------------|--------|--------|--------|-------------------------------------------------|
| frontend_api | ok     | –      | 540 ms | dashboard payload has 41 top-level keys         |
| agent_brief  | ok     | –      | 3 ms   | brief: 7 top sections · 11,495 chars rendered   |
| rag_index    | ok     | –      | 18 ms  | indexed 1355 facts · top-5 retrieval ok (tfidf) |
| ai_agent     | ok     | no     | 0 ms   | skipped (DATALENS_TEST_SKIP_LLM=1)              |

### Dataset: regression

6,035 rows · 18 cols · target = price

| MODULE       | STATUS | AVAIL. | TIME   | SUMMARY                                         |
|--------------|--------|--------|--------|-------------------------------------------------|
| frontend_api | ok     | –      | 605 ms | dashboard payload has 41 top-level keys         |
| agent_brief  | ok     | –      | 0 ms   | brief: 9 top sections · 12,671 chars rendered   |
| rag_index    | ok     | –      | 14 ms  | indexed 1155 facts · top-5 retrieval ok (tfidf) |
| ai_agent     | ok     | no     | 0 ms   | skipped (DATALENS_TEST_SKIP_LLM=1)              |

### Dataset: no\_target

4,035 rows · 22 cols · target = none

| MODULE       | STATUS | AVAIL. | TIME   | SUMMARY                                         |
|--------------|--------|--------|--------|-------------------------------------------------|
| frontend_api | ok     | –      | 303 ms | dashboard payload has 41 top-level keys         |
| agent_brief  | ok     | –      | 0 ms   | brief: 8 top sections · 12,620 chars rendered   |
| rag_index    | ok     | –      | 16 ms  | indexed 1271 facts · top-5 retrieval ok (tfidf) |
| ai_agent     | ok     | no     | 0 ms   | skipped (DATALENS_TEST_SKIP_LLM=1)              |

## 11 Timing budget

End-to-end pipeline time is dominated by three things: the ML leaderboard (cross-validated training of  $\sim 6$  estimators), SHAP explainability (which needs to instrument the winner), and the LLM agent call (which depends on network round-trips when Ollama Cloud is the backend). The deep-statistics phase can also be a non-trivial slice on wide datasets because it has to walk every numeric and categorical pair within its budget.

| DATASET        | ROWS  | COLS | PIPELINE | COMPONENTS TOTAL | TOP-3 SLOWEST                                                                              |
|----------------|-------|------|----------|------------------|--------------------------------------------------------------------------------------------|
| classification | 5,060 | 21   | 17.50 s  | 26.83 s          | explainability (7.84 s),<br>model_leaderboard<br>(7.44 s), drift_analysis<br>(4.46 s)      |
| regression     | 6,035 | 18   | 6.52 s   | 14.73 s          | drift_analysis (7.16 s),<br>deep_statistics_v2 (3.35<br>s), pdf_report_builder<br>(1.58 s) |
| no_target      | 4,035 | 22   | 4.87 s   | 7.97 s           | drift_analysis (2.66 s),<br>deep_statistics_v2 (2.02<br>s), pdf_report_builder<br>(1.51 s) |

## 12 Limitations and future work

**Modeling depth.** The leaderboard is intentionally conservative: a small set of well-understood estimators with default hyperparameters and no automated tuning. This keeps the runtime under five seconds on most datasets, but it also means DataLens does not pretend to crown a state-of-the-art model. Users who want that should treat the leaderboard as a baseline and continue in their own modeling environment.

**Drift baseline.** Drift is computed against a user-supplied reference file (or, in our component test, against a 50/50 row split of the same dataset). DataLens does not yet store a canonical historical baseline, so the drift surface is reactive rather than proactive. Persistent “time-zero” baselines would close that gap.

**Causality.** Every relationship reported here — correlation, SHAP, drift — is associational, not causal. DataLens deliberately does not embed causal claims in any output, but a downstream user is free to mistake “ $x$  is the most important SHAP feature” for “ $x$  causes  $y$ ”. This is a common failure mode in ML reporting, and it warrants explicit disclaimers in the dashboard.

**LLM grounding.** The dataset brief is the agent’s most accurate source of truth, but the brief itself is a lossy compression of the full analysis result. Questions whose answer requires evidence the brief did not include will fall back to RAG retrieval over the fact index, which is more flexible but lower precision. Future work includes letting the agent *re-call* into the dataset (re-loading the dataframe and asking pandas) when retrieval comes back empty.

**Scaling.** The pipeline holds the entire dataset in memory. For datasets larger than a few million rows, the more expensive phases (deep statistics, anomaly ensemble, leaderboard) start to noticeably hurt. Streaming or sampled variants are planned.

**Internationalisation.** The text profile uses NLTK’s English stopword list when available;

everything else is dtype-driven and language-agnostic. Language-aware tokenisation, stopwords lists, and stemming are not currently in scope.

## References

## References

- [1] Lundberg, S. M. & Lee, S.-I. (2017). *A Unified Approach to Interpreting Model Predictions*. Advances in Neural Information Processing Systems (NeurIPS), 30.
- [2] Breiman, L. (2001). *Random Forests*. Machine Learning, 45(1), 5–32.
- [3] Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). *Isolation Forest*. Proceedings of the 8th IEEE International Conference on Data Mining (ICDM), 413–422.
- [4] Breunig, M. M., Kriegel, H.-P., Ng, R. T., & Sander, J. (2000). *LOF: Identifying Density-Based Local Outliers*. ACM SIGMOD Record, 29(2), 93–104.
- [5] Cleveland, R. B., Cleveland, W. S., McRae, J. E., & Terpenning, I. (1990). *STL: A Seasonal-Trend Decomposition Procedure Based on Loess*. Journal of Official Statistics, 6(1), 3–73.
- [6] Hyndman, R. J. & Athanasopoulos, G. (2018). *Forecasting: Principles and Practice* (2nd ed.). OTexts.
- [7] Karakoulas, G. (2004). *Empirical Validation of Retail Credit-Scoring Models*. Industry working paper, popularised in credit-risk monitoring.
- [8] Kolmogorov, A. N. (1933). *Sulla determinazione empirica di una legge di distribuzione*. Giornale dell’Istituto Italiano degli Attuari, 4, 83–91.
- [9] Dickey, D. A. & Fuller, W. A. (1979). *Distribution of the Estimators for Autoregressive Time Series with a Unit Root*. Journal of the American Statistical Association, 74(366a), 427–431.
- [10] Kwiatkowski, D., Phillips, P. C. B., Schmidt, P., & Shin, Y. (1992). *Testing the Null Hypothesis of Stationarity Against the Alternative of a Unit Root*. Journal of Econometrics, 54(1–3), 159–178.
- [11] Cramér, H. (1946). *Mathematical Methods of Statistics*. Princeton University Press.
- [12] Shapley, L. S. (1953). *A Value for  $n$ -Person Games*. Contributions to the Theory of Games, 2, 307–317.

## A Per-component test table

This table mirrors the manifest 1:1, sorted by dataset and then by category.

**Dataset: classification**

| MODULE              | STATUS | AVAIL. | TIME   | SUMMARY                                                                                                                                                                   |
|---------------------|--------|--------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| loader              | ok     | –      | 0 ms   | loaded 5,060 rows × 21 cols from customer_churn.csv                                                                                                                       |
| profiler            | ok     | –      | 21 ms  | profile: 9 numeric, 12 categorical, 2 date · 4,763 missing cells                                                                                                          |
| column_roles        | ok     | –      | 8 ms   | roles: id=0, price=['monthly_charges', 'total_charges', 'feedback_text'], volume=[], target_candidates=['gender', 'region', 'plan', 'contract_type', 'payment_method']... |
| dataset_signals     | ok     | –      | 34 ms  | signals on: has_numeric_columns, has_multiple_numeric_columns, has_categorical_columns, has_datetime_columns, has_text_columns, has_long_text_columns                     |
| analysis_selector   | ok     | –      | 34 ms  | selected=27 · recommended=0 · skipped=1                                                                                                                                   |
| health_score        | ok     | –      | 24 ms  | health = 98.21/100 (Excellent)                                                                                                                                            |
| ml_readiness        | ok     | –      | 2 ms   | ML readiness = 74.33/100 (Mostly Ready)                                                                                                                                   |
| advanced_indicators | ok     | –      | 58 ms  | utility ranked 21 cols · top anomaly score 0.00                                                                                                                           |
| signal_engine       | ok     | –      | 83 ms  | 8 signals · Low:3, None:1, Medium:2, Informational:2                                                                                                                      |
| cleaner             | ok     | –      | 86 ms  | cleaning 98.21 → 99.78 · actions=7                                                                                                                                        |
| deep_statistics_v2  | ok     | yes    | 2.78 s | pairs: num=20, cat=20, groupdiff=20                                                                                                                                       |
| anomaly_ensemble    | ok     | yes    | 1.31 s | available=True · top rows=25 · detectors=0                                                                                                                                |
| model_leaderboard   | ok     | yes    | 7.44 s | task=classification · winner=LogisticRegression · score=0.8745                                                                                                            |
| explainability      | ok     | yes    | 7.84 s | method=shap.LinearExplainer · top features: feedback_text, days_since_last_login, contract_type                                                                           |
| time_series         | ok     | yes    | 177 ms | date=num_support_tickets · numeric=churned · freq=irregular                                                                                                               |
| text_profile        | ok     | yes    | 75 ms  | profiled 1 text col(s): feedback_text                                                                                                                                     |
| drift_analysis      | ok     | yes    | 4.46 s | shared cols=21 · drifted=4 · severe=3 · verdict=severe                                                                                                                    |
| chart_planner       | ok     | –      | 31 ms  | planned 25 charts (22 distinct types)                                                                                                                                     |

|                    |    |    |        |                                                                  |
|--------------------|----|----|--------|------------------------------------------------------------------|
| visualizer         | ok | –  | 0 ms   | rendered 23 charts to /static/charts/                            |
| pdf_report_builder | ok | –  | 1.80 s | PDF written:<br>comp_test_classification_report.pdf<br>(1366 KB) |
| frontend_api       | ok | –  | 540 ms | dashboard payload has 41 top-level keys                          |
| agent_brief        | ok | –  | 3 ms   | brief: 7 top sections · 11,495 chars rendered                    |
| rag_index          | ok | –  | 18 ms  | indexed 1355 facts · top-5 retrieval ok (tfidf)                  |
| ai_agent           | ok | no | 0 ms   | skipped (DATALENS_TEST_SKIP_LLM=1)                               |

### Dataset: regression

| MODULE              | STATUS | AVAIL. | TIME   | SUMMARY                                                                                                                                                                              |
|---------------------|--------|--------|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| loader              | ok     | –      | 0 ms   | loaded 6,035 rows × 18 cols from real_estate.csv                                                                                                                                     |
| profiler            | ok     | –      | 18 ms  | profile: 9 numeric, 9 categorical, 1 date · 650 missing cells                                                                                                                        |
| column_roles        | ok     | –      | 6 ms   | roles: id=0, price=['days_on_market', 'price'], volume=['lot_size'], target_candidates=['neighborhood', 'property_type', 'zoning', 'bedrooms', 'floors', 'has_garage', 'has_pool'... |
| dataset_signals     | ok     | –      | 28 ms  | signals on: has_numeric_columns, has_multiple_numeric_columns, has_categorical_columns, has_datetime_columns, has_text_columns, has_long_text_columns                                |
| analysis_selector   | ok     | –      | 28 ms  | selected=20 · recommended=0 · skipped=8                                                                                                                                              |
| health_score        | ok     | –      | 21 ms  | health = 99.52/100 (Excellent)                                                                                                                                                       |
| ml_readiness        | ok     | –      | 2 ms   | ML readiness = 80.82/100 (Mostly Ready)                                                                                                                                              |
| advanced_indicators | ok     | –      | 161 ms | utility ranked 18 cols · top anomaly score 0.00                                                                                                                                      |
| signal_engine       | ok     | –      | 184 ms | 8 signals · Low:4, None:1, Medium:1, Informational:2                                                                                                                                 |
| cleaner             | ok     | –      | 79 ms  | cleaning 99.52 → 99.79 · actions=4                                                                                                                                                   |
| deep_statistics_v2  | ok     | yes    | 3.35 s | pairs: num=20, cat=15, groupdiff=20                                                                                                                                                  |



|                    |    |     |        |                                                        |
|--------------------|----|-----|--------|--------------------------------------------------------|
| anomaly_ensemble   | ok | yes | 428 ms | available=True · top rows=25 · detectors=0             |
| model_leaderboard  | ok | yes | 385 ms | task=regression · winner=None · score=None             |
| explainability     | ok | no  | 239 ms | skipped — no leaderboard winner                        |
| time_series        | ok | yes | 337 ms | date=sale_date · numeric=price · freq=irregular        |
| text_profile       | ok | yes | 81 ms  | profiled 1 text col(s): description                    |
| drift_analysis     | ok | yes | 7.16 s | shared cols=18 · drifted=3 · severe=3 · verdict=severe |
| chart_planner      | ok | —   | 23 ms  | planned 23 charts (19 distinct types)                  |
| visualizer         | ok | —   | 0 ms   | rendered 21 charts to /static/charts/                  |
| pdf_report_builder | ok | —   | 1.58 s | PDF written: comp_test_regression_report.pdf (1237 KB) |
| frontend_api       | ok | —   | 605 ms | dashboard payload has 41 top-level keys                |
| agent_brief        | ok | —   | 0 ms   | brief: 9 top sections · 12,671 chars rendered          |
| rag_index          | ok | —   | 14 ms  | indexed 1155 facts · top-5 retrieval ok (tfidf)        |
| ai_agent           | ok | no  | 0 ms   | skipped (DATALENS_TEST_SKIP_LLM=1)                     |

### Dataset: no\_target

| MODULE            | STATUS | AVAIL. | TIME  | SUMMARY                                                                                                                                                                        |
|-------------------|--------|--------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| loader            | ok     | —      | 0 ms  | loaded 4,035 rows × 22 cols from medical_records.csv                                                                                                                           |
| profiler          | ok     | —      | 18 ms | profile: 12 numeric, 10 categorical, 1 date · 905 missing cells                                                                                                                |
| column_roles      | ok     | —      | 7 ms  | roles: id=0, price=['heart_rate'], volume=['medication_count'], target_candidates=['gender', 'ethnicity', 'diagnosis_code', 'diagnosis_label', 'smoker', 'medication_count'... |
| dataset_signals   | ok     | —      | 29 ms | signals on: has_numeric_columns, has_multiple_numeric_columns, has_categorical_columns, has_datetime_columns, has_text_columns, has_missing_values                             |
| analysis_selector | ok     | —      | 28 ms | selected=20 · recommended=0 · skipped=8                                                                                                                                        |

|                     |    |     |        |                                                                            |
|---------------------|----|-----|--------|----------------------------------------------------------------------------|
| health_score        | ok | –   | 25 ms  | health = 99.42/100 (Excellent)                                             |
| ml_readiness        | ok | –   | 2 ms   | ML readiness = 78.11/100 (Mostly Ready)                                    |
| advanced_indicators | ok | –   | 43 ms  | utility ranked 22 cols · top anomaly score 0.00                            |
| signal_engine       | ok | –   | 66 ms  | 8 signals · Low:4, None:1, Medium:1, Informational:2                       |
| cleaner             | ok | –   | 91 ms  | cleaning 99.42 → 99.89 · actions=5                                         |
| deep_statistics_v2  | ok | yes | 2.02 s | pairs: num=20, cat=20, groupdiff=20                                        |
| anomaly_ensemble    | ok | yes | 522 ms | available=True · top rows=25 · detectors=0                                 |
| model_leaderboard   | ok | no  | 0 ms   | skipped — no target column provided                                        |
| explainability      | ok | no  | 0 ms   | skipped — no target column provided                                        |
| time_series         | ok | yes | 562 ms | date=num_visits_past_year · numeric=days_since_last_visit · freq=irregular |
| text_profile        | ok | yes | 38 ms  | profiled 1 text col(s): clinical_notes                                     |
| drift_analysis      | ok | yes | 2.66 s | shared cols=22 · drifted=2 · severe=2 · verdict=severe                     |
| chart_planner       | ok | –   | 23 ms  | planned 20 charts (17 distinct types)                                      |
| visualizer          | ok | –   | 0 ms   | rendered 19 charts to /static/charts/                                      |
| pdf_report_builder  | ok | –   | 1.51 s | PDF written: comp_test_no_target_report.pdf (1223 KB)                      |
| frontend_api        | ok | –   | 303 ms | dashboard payload has 41 top-level keys                                    |
| agent_brief         | ok | –   | 0 ms   | brief: 8 top sections · 12,620 chars rendered                              |
| rag_index           | ok | –   | 16 ms  | indexed 1271 facts · top-5 retrieval ok (tfidf)                            |
| ai_agent            | ok | no  | 0 ms   | skipped (DATALENS_TEST_SKIP_LLM=1)                                         |

## B Failure traces

No failures were recorded in this audit run.